

MeetNote AI — Project Documentation & Code Flow

AI-powered, accessibility-first meeting assistant: upload a meeting recording → get a timestamped transcript, summary, action items, mind map, sentiment, translation and accessibility-friendly views.

Team (TCS): Ekta Tajne (3193218) · Dhanashree Hiwase (3193205) · Sharyu Mate (3202469) · Yash Sathavane (3193223) · Anjali Kumari (3193228)

1. Overview

1.1 Problem

Meetings are inefficient and not inclusive. People spend time taking notes instead of participating; fast conversations are hard for hearing-impaired users, non-native speakers and people who struggle to keep up. Meetings are rarely searchable and action items get lost.

1.2 Solution

MeetNote AI converts meeting audio to text with **OpenAI Whisper (running locally)**, then builds structured insights on top of it and presents them in dashboards tailored to different accessibility needs.

1.3 Objectives → what implements them

Objective (from presentation)	Where it lives in code
Speech → accurate text (Whisper)	backend/main.py → /transcribe/ (model.transcribe)
Timestamped transcripts	build_timeline() → segments with start , end , mm:ss
Speaker identification	build_timeline() (heuristic, see §7)
Action items, decisions, risks, questions	semantic_analysis()
Summary & mind map	generate_insights() + Mermaid in MeetingAnalysisView.jsx
Searchable / documented meetings	Meeting history in browser localStorage , PDF export
Accessibility	4 profile dashboards + global font/contrast/motion controls
Non-native speakers	Translation via deep-translator (/translate-text/ , target_lang)

2. Implemented vs. Presented (important for demo / viva)

The presentation describes the *target* architecture. The code in the zip implements part of it. Being clear about this avoids surprises when someone reads the code.

Item in presentation	Status in code	Reality
React + Vite + Tailwind	Implemented	React 19, Vite 8, Tailwind 3
Shadcn UI	Not present	Custom Tailwind components (no shadcn in package.json)
Whisper transcription	Implemented	Local whisper model "base"
Backend "Flask + SQLAlchemy + SQLite"	Different	FastAPI , no database
JWT auth / 2FA	Not implemented	Login form only calls navigate("/dashboard")
Pyannote speaker diarization	Not implemented	Simple rule: pause > 1.2 s ⇒ next speaker (A/B/C cycle)
Google Gemini "Strategic Analyst"	Not implemented	Summary/insights use rule-based NLP in Python
VADER / TextBlob sentiment	Not implemented	Small positive/negative word lists
Deep-Translator	Implemented	GoogleTranslator (needs internet)
Mermaid.js mind map	Implemented	Loaded from CDN, rendered in

		browser
ISL avatar signing	Prototype	Canvas-drawn avatar (AvatarCanvas.jsx) driven by sentence text
Live meeting analysis	Partial	Records mic + level meter; downloads .webm ; no live transcription
Video analysis	Mock	MeetingVideoPage uses mockData.js
Google Calendar / auto email	Not implemented	Roadmap ("Week 4") only
Emotion detection	Text-based only	Per-segment sentiment label; video emotions are mock

3. Architecture

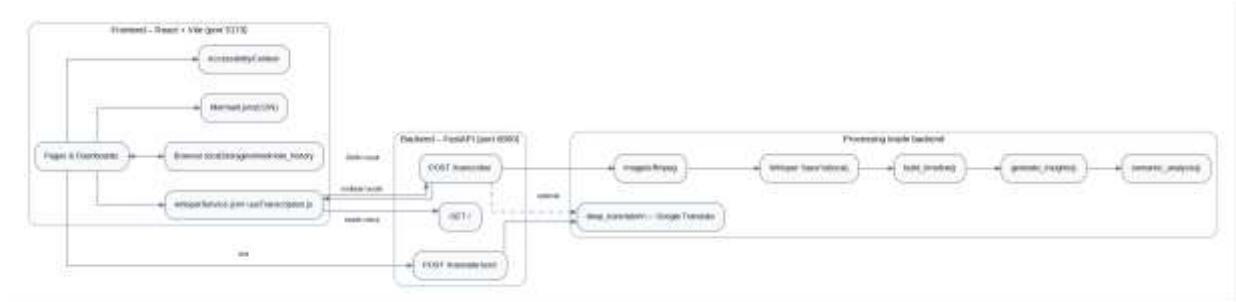


Figure 1. MeetNote AI system architecture

Style: simple client-server. All AI processing is synchronous inside one request. State is kept in React state + browser localStorage; the server stores nothing.

4. Project Structure

```

MeetNote 02/
├── backend/
│   ├── main.py          # FastAPI app - the real backend (456 lines)
│   ├── app.py           # Standalone Whisper test script (not part of the API)
│   ├── transcription.txt # Sample output from app.py
│   └── .env              # Environment file (contains an API key - see §11)
├── frontend/
│   ├── index.html · vite.config.js · tailwind.config.js · package.json
│   ├── .env              # VITE_API_BASE_URL
│   └── src/
│       ├── main.jsx · App.jsx          # Entry + routes
│       ├── context/AccessibilityContext.jsx
│       ├── services/whisperService.js  # fetch wrappers to backend
│       ├── hooks/useTranscription.js    # loading/error/result state
│       ├── api.js                      # older helper (unused)
│       ├── data/mockData.js            # mock video-analysis data
│       ├── pages/
│       │   ├── LoginPage.jsx
│       │   ├── DashboardHome.jsx      # "Choose a Source" hub
│       │   ├── MeetingAudioPage.jsx   # upload audio → analysis
│       │   ├── MeetingVideoPage.jsx   # video + sign language (mock data)
│       │   ├── MeetingLivePage.jsx    # mic recorder
│       │   ├── MeetingAnalysisView.jsx # main results screen (700 lines)
│       │   └── dashboards/            # Default, ColorBlind, Dyslexia, Hearing, ADHD
│       └── components/
│           ├── layout/ (DashboardShell, Navbar, Sidebar, MeetingHistorySidebar)
│           └── shared/ (UploadZone, ResultTabs, TranscriptPanel, SummaryPanel, MindMap)

```

```

├─ meeting/ (MeetingAnalysisView, SignLanguageInterpretation)
├─ AvatarCanvas.jsx # canvas avatar

```

Files that appear unused (not imported anywhere): components/UploadPanel.jsx , HeroSection.jsx , StatusBar.jsx , api.js , data/mockTranscript.js , and the older top-level components/Navbar.jsx / TranscriptPanel.jsx .

5. Tech Stack (as actually used)

Layer	Technology
Frontend	React 19, React Router 7, Vite 8, Tailwind CSS 3
Visualisation	Mermaid.js 10 (CDN), HTML Canvas (avatar)
Backend	Python, FastAPI, Uvicorn, Pydantic
Speech-to-text	OpenAI Whisper (base , local)
Audio decoding	imageio-ffmpeg (ffmpeg binary injected into PATH)
Translation	deep-translator (GoogleTranslator)
Text analytics	Pure Python (re , collections.Counter)
Browser APIs	MediaRecorder , getUserMedia , AudioContext , speechSynthesis , localStorage

6. End-to-End Code Flow (audio upload — the main path)

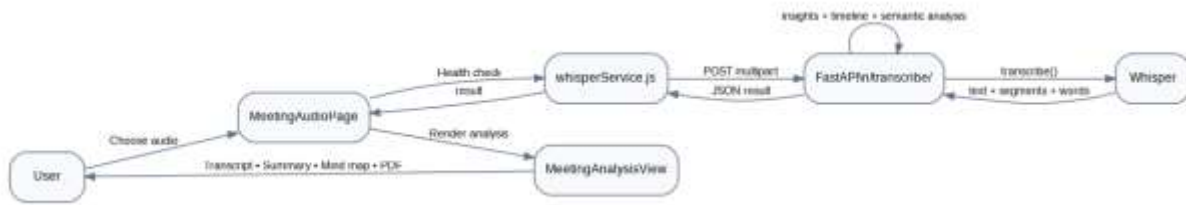


Figure 2. End-to-end audio analysis flow

6.1 Backend flow, step by step (POST /transcribe/)

1. **Receive** – file (UploadFile), mode (transcribe | translate), target_lang (form fields).
2. **Save** the upload as temp_<filename> in the working directory.
3. **Whisper** – model.transcribe(path, task, word_timestamps=True)

mode="translate" → Whisper's own *translate-to-English* task.

mode="transcribe" → text in the spoken language + detected language code.

4. **Optional translation** – if target_lang is given and differs from the detected language, translate_text_local() translates the full text (chunked at 4 500 chars for Google's limit).
5. **generate_insights(text)** → summary[] and mindmap{} (see §7.2).
6. **build_timeline(result)** → segments[], speaker_turns[], words[] (see §7.1).
7. **Batch-translate** segments, turns, mind-map labels and summary lines with translate_batch() (one API call per group, using an invisible separator string) when translation was requested.
8. **semantic_analysis(text, segments)** → topics, decisions, action items, questions, risks, sentiment (see §7.3). Note: runs on the (possibly translated) text.
9. **Cleanup** – temp file removed (also on error) and JSON returned.

6.2 Frontend flow

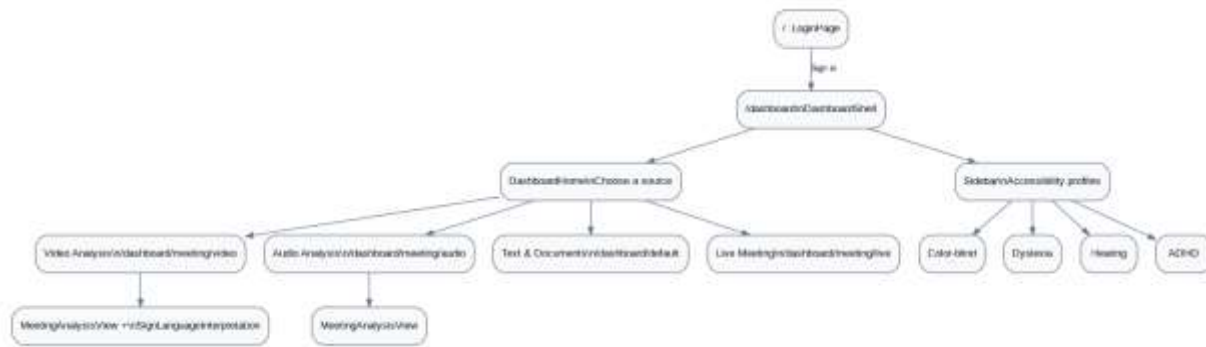


Figure 3. Frontend navigation flow

- App.jsx wraps everything in AccessibilityProvider + BrowserRouter; /dashboard is a layout route (DashboardShell) with nested pages rendered through <Outlet />.
- DashboardShell hides the Navbar/Sidebar on meeting pages and the default dashboard to give full-screen space.
- Unknown URLs redirect to /.

7. Backend Logic in Detail

7.1 build_timeline(result) — timestamps & "speakers"

- Iterates Whisper segments; skips empty ones.
- **Speaker heuristic:** if the silence before a segment is > 1.2 s, the speaker index increments; label = Speaker A/B/C (index % 3).
- Produces three lists: segments (seg-N), speaker_turns (turn-N), words (per-word start/end).
- format_mmss() turns seconds into MM:SS.

□ This is not true diarization — it doesn't analyse voices. Pyannote (shown in the architecture slide) is future work.

7.2 generate_insights(text) — summary & mind map (extractive, rule-based)

10. Split into sentences (re.split on . ? !).
11. Keep words of 4+ letters, remove a stop-word list, count frequencies (Counter).
12. **Summary:** score each sentence = (sum of its word frequencies) ÷ (words + 1); pick the **top 3** and keep original order.
13. **Mind map:** center = most frequent keyword; next 3 keywords become branches (colors cyan/amber/red); each branch's children = 3 most frequent co-occurring words in sentences containing that keyword.

7.3 semantic_analysis(text, segments) — rule-based NLP

Output	Rule
questions	sentence contains ?
action_items	contains markers: need to, please, let us, we should, I will, we will, follow up, schedule, send, share
decisions	decided, we agree, agreed, approved, finalize, confirmed, locked, conclude
risks	risk, blocker, issue, problem, delay, stuck, concern, unknown, dependency
topics	6 most common meaningful words
sentiment.overall	(positive words – negative words) over first 20 sentences → Positive ≥ 2, Negative ≤ -2, else Neutral
sentiment.segments	same scoring for first 12 segments, with timestamp

Results are de-duplicated and capped at 5 per category.

7.4 Translation helpers

- `translate_text_local()` – maps language names → ISO codes, uses `GoogleTranslator(source, target)`, chunks long text; on failure returns "[LANG] original text".
- `translate_batch(texts, ...)` – joins many strings with an invisible separator (`\u2063\u2064\u2063`), translates in one call ($\leq 400\text{--}4\,500$ chars per call), splits back, pads/trims to original length; on error returns the originals.
- Supported names in map: English, Hindi, Spanish, French, German, Marathi, Gujarati, Bengali, Tamil, Telugu, Kannada, Malayalam, Punjabi (others pass through as ISO codes).

7.5 Startup details

- `imageio_ffmpeg` provides `ffmpeg`; its folder is appended to `PATH` so `Whisper` can decode audio without a system install.
- `Whisper` model is loaded **once at import** (`whisper.load_model("base")`) — first start is slow, requests are faster.
- `CORS` is open (`allow_origins=["*"]`).

8. API Reference

GET /

Health check. → `{"message": "Local Whisper Backend running successfully 🌱"}`

POST /transcribe/ (multipart/form-data)

Field	Type	Default	Notes
file	file	required	any audio <code>ffmpeg</code> can decode
mode	string	transcribe	translate → <code>Whisper</code> translates to English
target_lang	string	""	ISO code or name, e.g. <code>hi</code> , <code>Marathi</code>

Response

```
{
  "mode": "transcribe",
  "language": "en",
  "target_lang": "hi",
  "text": "full transcript...",
  "summary": ["sentence 1", "sentence 2", "sentence 3"],
  "mindmap": {
    "center": "Topic",
    "nodes": [{ "label": "Keyword", "color": "cyan", "children": ["a", "b", "c"] }]
  },
  "segments": [{ "id": "seg-0", "speaker": "Speaker A", "start": 0.0, "end": 4.2, "timestamp": "00:00",
"text": "..."}],
  "speaker_turns": [{ "id": "turn-0", "speaker": "Speaker A", "start": 0.0, "end": 4.2, "timestamp": "00:00",
"text": "..."}],
  "words": [{ "id": "w-0-0", "word": "Hello", "start": 0.0, "end": 0.4, "timestamp": "00:00", "speaker":
"Speaker A"}],
  "semantic": {
    "topics": [], "decisions": [], "action_items": [], "questions": [], "risks": [],
    "sentiment": { "overall": { "label": "Neutral", "score": 0 }, "segments": [{ "timestamp": "00:00", "label":
"Neutral", "score": 0 }] }
  }
}
```

Errors → `500 {"detail": "<message>"}`.

POST /translate-text/ (JSON)

```
{ "text": "...", "source_lang": "en", "target_lang": "hi" }
```

```
→ { "translated_text": "...", "source_lang": "en", "target_lang": "hi" }
```

□ The frontend also defines `transcribeFromUrl()` which calls `POST /transcribe-url/`, but that endpoint does not exist in the backend.

9. Frontend Logic in Detail

9.1 Data layer

File	Role
services/whisperService.js	<code>checkBackendHealth()</code> , <code>transcribeAudio(file, {mode, target_lang})</code> , <code>transcribeFromUrl()</code> ; base URL = <code>VITE_API_BASE_URL</code> or <code>http://127.0.0.1:8000</code>
hooks/useTranscription.js	Holds result, loading, error; <code>transcribeFile</code> , <code>transcribeRecording</code> (wraps a Blob as recording.webm), <code>transcribeUrl</code>

9.2 Accessibility system

`AccessibilityContext` stores `activeProfile`, `fontSize`, `highContrast`, `reduceMotion`. A `useEffect` applies them globally: root font size in px and CSS classes on `<body>` (`high-contrast`, `reduce-motion`, `profile-<name>`). The Sidebar sets the profile and exposes font size / contrast / motion controls.

Profile dashboard	What it adapts
Color-blind	No colour-only signals; distinct shapes, high-contrast palette
Dyslexia	Font choice (Arial / "OpenDyslexic style"), letter spacing 0.08em, wide line spacing, soft palettes (e.g. Soft Cream)
Hearing-impaired	Visual sound-alert area, transcripts, two-way text box that speaks via <code>speechSynthesis</code>
ADHD	Focus timer, transcript chunked into 1 / 3 / 5-sentence groups
Default	Standard upload + tabs

All five use the shared components: `UploadZone` → `ResultTabs` → `TranscriptPanel` / `SummaryPanel` / `MindMap`.

9.3 MeetingAnalysisView.jsx (results screen)

- Builds `transcriptTurns` from `speaker_turns/segments`.
- **Insights** shown from semantic (decisions, actions, questions, risks, sentiment).
- **Mind map:** `buildMermaidMindmap()` converts the JSON into Mermaid mindmap syntax; Mermaid is injected from `cdn.jsdelivr.net` at runtime and rendered to SVG (with a raw-code fallback).
- **Translation:** calls `POST /translate-text/` on the full transcript for a chosen language.
- **History:** each analysis is saved in `localStorage["meetnote_history"]`; `MeetingHistorySidebar` lists and reloads them.
- **PDF export:** builds an HTML report (summary, transcript, mind map SVG) in a new window and calls `window.print()`.

9.4 Other pages

Page	Behaviour today
MeetingAudioPage	Real flow (section 6) with translation option and health check
MeetingLivePage	<code>getUserMedia</code> + <code>MediaRecorder</code> , timer, audio-level meter via <code>AudioContext</code> / <code>AnalyserNode</code> ; on stop downloads a .webm — not transcribed

MeetingVideoPage	Uses mockMeetingVideoAnalysis (mock speakers/emotions); includes SignLanguageInterpretation (tabs: Mind Map / Emotion / Speaker) and a PDF export
LoginPage	UI only; submit navigates to /dashboard

9.5 Sign-language avatar

AvatarCanvas.jsx draws a stick-figure/limb avatar on a <canvas>, using poses and interpolation (lp, setPose, limb) animated per sentence chunk (sentenceChunk prop, pausable). It is a visual prototype, not a real ISL translator.

10. Setup & Run

Prerequisites

Python 3.9+, Node.js 18+, internet (for translation and Mermaid CDN).

Backend

The zip has no requirements.txt; these are the imports the code needs:

```
cd backend
python -m venv .venv
# Windows: .venv\Scripts\activate      macOS/Linux: source .venv/bin/activate
pip install fastapi "uvicorn[standard]" python-multipart pydantic \
    openai-whisper imageio-ffmpeg deep-translator
python -m uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

First run downloads the Whisper base model (~140 MB). Check <http://127.0.0.1:8000/docs>.

Frontend

```
cd frontend
npm install
# .env → VITE_API_BASE_URL=http://127.0.0.1:8000
npm run dev
```

Open the URL Vite prints (usually <http://localhost:5173>) → Sign in → Audio Analysis → upload a file.

Environment variables

File	Variable	Used?
frontend/.env	VITE_API_BASE_URL	☐ backend base URL
backend/.env	OPENAI_API_KEY	☐ not read by any code (Whisper runs locally)

11. Known Issues, Risks & Improvements

Security / hygiene

- backend/.env in the zip contains an API key. It isn't used — **delete it and rotate the key** if it's real; add .env to .gitignore.
- No authentication on the API; CORS allows all origins; uploads are written to the working directory with the client filename (use tempfile + sanitised names).

Functional gaps

- Login/JWT/2FA not implemented (login is cosmetic).

- Speaker labels are pause-based guesses; summary/action items are keyword rules (can miss context, and English-only word lists apply even after translation).
- `/transcribe-url/` missing in backend; Live page doesn't transcribe; Video page is mock data.
- `translate_text_local` failure returns "[HI] original" silently — the UI can't tell it failed.
- Duration in the analysis view is estimated ($\text{turns} \times 0.5 \text{ min}$), not real audio length.
- Processing is synchronous; long audio blocks the request (no progress or job queue).
- Dashboards pass an `apiKey` to `useTranscription` that the hook ignores.

Suggested roadmap (aligned with your slides)

14. Real diarization (pyannote.audio) and an LLM (Gemini) for summary/action items.
15. Auth: JWT + user accounts; persist meetings in SQLite/PostgreSQL via SQLAlchemy.
16. Live transcription via chunked streaming (WebSocket) into Whisper.
17. Proper ISL/avatar pipeline; Google Calendar + auto-email of action items.
18. Add tests, `requirements.txt`, Dockerfile.

12. Quick Viva / Demo Answers

- **Why Whisper locally?** Privacy, no per-request cost, works offline for transcription.
- **How are timestamps produced?** Whisper returns per-segment/word times; `build_timeline` formats them as `MM:SS`.
- **How do you find action items?** Sentence-level keyword rules (§7.3) — designed to be swapped for an LLM.
- **How is accessibility handled?** Global context (font size, contrast, motion) + four profile-specific dashboards.
- **What does the mind map use?** Top keywords → JSON → Mermaid mindmap syntax → SVG in browser.
- **Where is data stored?** Only in the browser (`localStorage`); the server is stateless.